

SAATHI — Backend & API Specification

FastAPI implementation contract for OpenCode. Derived from the supplied TRD/PRD and extended only where required to support the supplied UI flow.

1. Backend Architecture

```

React + TypeScript + Tailwind
↓ Axios
FastAPI application
↓
████████████████████████████████████████████████████████████████████████████████
DB Local ML Gemini
↓ ↓ ↓
users wellbeing AI support
cases inference extraction
support
audit

```

The MVP is a modular monolith: one FastAPI application, one database and one frontend. Do not add Kafka, Redis, microservices or a message broker.

2. Technology Stack

Layer	Technology
Frontend API boundary	Axios
Backend	Python + FastAPI
ORM	SQLAlchemy
Validation	Pydantic
MVP database	SQLite
Production path	PostgreSQL + migrations
Auth	JWT + password hashing
ML	scikit-learn + joblib
GenAI	google-genai, server-side
Files	Local filesystem for MVP
Testing	pytest, FastAPI TestClient, Vitest

3. Environment Variables

```
DATABASE_URL=sqlite:///./saathi.db
JWT_SECRET_KEY=<long-random-secret>
GEMINI_API_KEY=<server-side-secret>
GEMINI_MODEL=<configurable-model>
UPLOAD_DIR=./uploads
MAX_UPLOAD_MB=10
CORS_ORIGINS=http://localhost:5173
```

Never commit .env, API keys, authkeys, JWT secrets, uploaded files or database files.

4. Repository Structure

```
saathi/
■■■ frontend/
```

```

■■■■ backend/
■ ■■■ app/
■ ■■■ main.py
■ ■■■ core/
■ ■■■ config.py
■ ■■■ security.py
■ ■■■ db/
■ ■■■ database.py
■ ■■■ models.py
■ ■■■ schemas/
■ ■■■ api/routes/
■ ■■■ services/
■ ■■■ ml_service.py
■ ■■■ gemini_service.py
■ ■■■ case_service.py
■ ■■■ priority_service.py
■ ■■■ models/
■ ■■■ uploads/
■ ■■■ tests/
■ ■■■ .env.example
■■■■ docs/

```

5. Database Schema

Table	Key fields
users	id, email, phone, password_hash, role, profile_completed, phone_verified_at, created_at
profiles	id, user_id, full_name, display_name, preferred_language, case_relationship, emergency_contact, safe_contact_method, consent_at
cases	id, owner_user_id, case_number, category, category_verified, stage, court_name, next_hearing, summary, protection_request_open, created_at, updated_at
case_documents	id, case_id, owner_user_id, filename, mime_type, storage_path, extracted_json, uploaded_at
checkins	id, user_id, case_id, text, predicted_class, predicted_label, confidence, created_at
support_requests	id, user_id, case_id, type, status, priority, explicit_human_request, created_at, resolved_at
audit_logs	id, actor_user_id, action, resource_type, resource_id, created_at, metadata_json
otp_verifications	id, user_id, target_phone, provider, status, created_at, verified_at, expires_at
notifications	id, user_id, type, title, body, read_at, created_at
case_assignments	id, case_id, assignee_id, assignment_type, reason, created_at
support_actions	id, support_request_id, actor_id, action, notes, created_at
ai_conversations / ai_messages	conversation ownership, messages, timestamps, safety state

6. API Contract

Method	Path	Purpose
POST	/api/v1/auth/signup	Create account / start verification state
POST	/api/v1/auth/login	Authenticate
GET	/api/v1/me	Current user + profile completion
POST	/api/v1/auth/otp/start	Start account-mobile OTP
POST	/api/v1/auth/otp/verify	Backend verification of provider token
POST	/api/v1/auth/otp/resend	Resend OTP

Method	Path	Purpose
POST	/api/v1/profile	Create one-time profile
PATCH	/api/v1/profile	Explicit profile update
GET	/api/v1/cases/me	Own cases
POST	/api/v1/cases/upload	Upload case document
GET	/api/v1/cases/{id}	Authorised case detail
POST	/api/v1/checkins	Save + run local ML
GET	/api/v1/checkins/me	Own check-in history
POST	/api/v1/ai/chat	Gemini support chat
POST	/api/v1/ai/case-extract	Optional document extraction
POST	/api/v1/support-requests	Create human support request
GET	/api/v1/admin/queue	Authorised deterministic queue
GET	/api/v1/admin/cases/{id}	Authorised admin case view
POST	/api/v1/admin/cases/{id}/priority-recalculate	Recalculate priority
GET	/api/v1/notifications	User notifications
POST	/api/v1/notifications/{id}/read	Mark notification read
GET	/health	Health check

7. MSG91 OTP

```

Browser
↓
MSG91 widget
↓
access token
↓
FastAPI
↓
POST https://control.msg91.com/api/v5/widget/verifyAccessToken
{
  "authkey": "<server-side MSG91_AUTHKEY>",
  "access-token": "<widget-token>"
}
↓
verified account mobile

```

- The account-mobile OTP is the signup verification mechanism.
- The frontend callback is not proof of verification.
- Only the backend's verified provider result activates the account.
- Emergency contact is a separate profile field and is not part of signup verification.
- Never place MSG91 authkey or credential-like tokenAuth values in React.

8. ML Service

```

class WellbeingService:
    load vectorizer once
    load LogisticRegression model once

    predict(text):
        X = vectorizer.transform([text])

```

```

probabilities = model.predict_proba(X)[0]
class_id = model.classes_[probabilities.argmax()]
confidence = float(probabilities.max())
label = LABEL_MAP.get(str(class_id))
return {
    "class_id": int(class_id),
    "label": label,
    "confidence": confidence
}

```

Verified artifact facts: LogisticRegression with classes [0,1,2,3,4,5], 5,000 input features; TF-IDF vectorizer with 5,000 vocabulary entries, unigram range (1,1), lowercase=True. The semantic labels must come from original training metadata. If unavailable, keep label null and do not invent names.

9. Risk Engine Separation

The emotion classifier is not a distress-risk model. Create a separate RiskEngine interface. Until a validated risk model or approved deterministic rule set exists, its state is NOT_CONFIGURED. Do not convert emotion probabilities into a fabricated clinical or safety score.

10. Gemini Service

```

from google import genai
client = genai.Client(api_key=settings.GEMINI_API_KEY)
response = client.models.generate_content(
    model=settings.GEMINI_MODEL,
    contents=prompt
)

```

- Server-side only.
- Calm, concise, non-judgmental.
- No mental-health diagnosis.
- No legal conclusions.
- No claim that a case/crime has been verified by AI.
- No claim of contacting police, counsellors or emergency services unless the backend actually performs that action.
- Immediate danger switches to the safety-support flow.
- Never reveal prompts, API keys, database details or internal scoring.

11. Priority Service

```

HIGH_PRIORITY_CATEGORIES = {
    "RAPE_OR_GANG_RAPE",
    "MURDER_GRIEVOUS_HURT_ARSON",
    "WITNESS_INTIMIDATION_OR_THREATS",
    "CASTE_BASED_VIOLENCE_FAMILY_AFFECTED"
}

score = 0
if verified_category in HIGH_PRIORITY_CATEGORIES: score += 70
if protection_request_open: score += 20
if explicit_human_help_request: score += 15
if wellbeing_review_flag: score += 10
score = min(score, 100)

```

Expose categories such as HIGH, STANDARD and REVIEW rather than presenting the number as a clinical or legal severity score.

12. RBAC

Role	Allowed scope
Victim	Own profile, own cases, own check-ins, own support requests
Counsellor	Assigned well-being records and authorised support context
District Officer	Authorised district case/service data
Administrator	Aggregates by default; authorised administrative views only

Every endpoint must enforce authorization on the backend. Hiding a React button is not security.

13. Upload Security

- Accept PDF, JPG and PNG only in MVP.
- Maximum 10 MB.
- Generate a random server-side storage filename.
- Never use the user filename as a filesystem path.
- Store owner_user_id and case_id with every document.
- Validate MIME type and extension.
- Do not execute uploaded files.
- Never return filesystem paths to the browser.

14. Error Contract

Condition	Response
Unauthorized	401
Forbidden	403
Invalid upload/request	400
Oversized upload	413
ML unavailable	503
Gemini unavailable	Graceful fallback; core local result remains available
Database unavailable	503

15. Security Requirements

- Prevent IDOR and cross-role access.
- Hash passwords.
- Do not put sensitive profile/case data inside JWT.
- Do not use localStorage as the security boundary.
- Rate-limit authentication and OTP endpoints.
- Use secure CORS configuration.
- Do not log passwords, OTPs, JWTs, MSG91 authkeys, Gemini keys or raw sensitive check-in text.
- Audit login, verification, profile changes, case assignment, support actions, privileged access and admin changes.

16. Testing Matrix

- Signup, login, logout and one-time profile tests.
- Victim cannot access another user's case/check-in/profile.
- OTP provider verification is tested with a mock.
- Case upload ownership and type/size validation tests.
- Fixed-sample ML inference test.
- Unknown ML label test.
- Gemini failure fallback test.
- Priority calculation tests for every rule.
- Audit log creation test.
- Frontend protected-route and role-route tests.
- Responsive UI and async-state tests.

17. Implementation Order

- 1 Project setup + database + authentication.
- 2 One-time profile.
- 3 React layout + protected routing.
- 4 My Case empty state + upload + case-details state.
- 5 ML check-in integration.
- 6 Gemini server integration.
- 7 Admin priority queue.
- 8 Counsellor/support views.
- 9 Security + tests.
- 10 Demo polish and final verification.

18. Production Boundary

SQLite is the MVP database. The supplied TRD identifies PostgreSQL as the later production path. Production deployment should use migrations, managed secrets, isolated file/object storage, secure transport and centralized audit/security monitoring. Do not add those operational systems merely to complicate the MVP.